

Lab 1: The Multi-Paradigm Tour

CS 379: Programming Languages (Fall 2026)

Dianelsa Chiru

Part A: The Proof Detective (50 Points)

1. The Compilation vs. Interpretation Dilemma (15 pts):

Classify each of your three implementations — C, Python, and Go — as either **ahead-of-time compiled**, **interpreted**, or **JIT-compiled**, and justify each classification in 1-2 sentences. Go is the trickiest of the three: it produces a native binary like C, but it also ships a managed runtime (goroutine scheduler, garbage collector) inside that binary. Explain why Go still counts as "compiled" for the purposes of startup latency even though it carries runtime machinery C does not.

- **C**

C is an ahead-of-time compiled language. GCC translates the entire source program into native machine code before execution, producing a standalone executable that can run directly on the CPU.

- **Python**

For this lab, Python is classified as an interpreted language. Python first converts the source code into bytecode, but the Python interpreter executes that bytecode instead of compiling it ahead of time into a standalone native executable.

- **Go**

It is also ahead-of-time compiled because go build produces a native standalone executable before the program runs. Although the binary includes a managed runtime with a goroutine scheduler and garbage collector, there is no separate interpreter or JIT compilation step at startup.

2. Pipeline Tracing (15 pts):

Trace the expression $\text{total} = 5 + 3 * 2$ through the full pipeline from raw text to physical execution, once for the **C** path and once for the **Python** path. For each path, list the intermediate representations the expression passes through in order (e.g., source text $\rightarrow$ tokens $\rightarrow$... $\rightarrow$

machine code executing on the CPU). Your two traces should visibly diverge at the point where compilation and interpretation part ways — mark that point explicitly and explain what happens on each side of it.

- **C Path**

Source text → tokens → AST → native machine code → executable binary → CPU

The lexer creates tokens and the parser builds an AST where $3*2$ is evaluated before $5+6$.

C then compiles the AST into native machine code and an executable, which the CPU runs. The final value is total = 11.

- **Python Path**

Source text → tokens → AST → Python bytecode → Python interpreter → CPU

Python follows the same first stages, but after the AST it produces bytecode instead of a standalone native executable. Python interprets the bytecode and the CPU performs the operations, giving total = 14.

- Point explicitly: After the AST, C generates native machine code and an executable, while Python produces bytecode that is executed by the Python interpreter.

3. Paradigm Classification (10 pts):

For each of the four paradigms introduced this week — **Imperative, Object-Oriented, Functional, and Logic** — write one line of pseudocode that is idiomatic of that paradigm for the task "square every number in a list." Then state which paradigm your Part B statistics calculator falls under in each of its three languages, and note whether that classification ever changes depending on which language features you used (e.g., a for loop vs. a map).

- **Imperative**

For each x in numbers: add $x * x$ to squares

- **Object-Oriented**

squares = NumberList(numbers).squareAll()

- **Functional**

squares = map(square, numbers)

- **Logic**

Squares(List, Result) :- maplist(square, List, Result)

4. **Startup Latency in the Real World (10 pts):**

Using your measured startup-latency numbers from Part B, explain why a serverless platform like AWS Lambda historically imposed a "cold start tax" on interpreted-language functions but not on compiled-language functions. Then explain, in one paragraph, what a JIT does differently from either extreme (cite PyPy or Node's V8 engine) — and why a CubeSat flight computer, unlike a Lambda function, would almost never want a JIT.

My measured cold-start times were C:9.15 MS, Go: 14.92 ms, and Python: 47.59 ms. Python had the highest startup cost because it must start an interpreter and execute bytecode, while C and Go already have native machine code available. This explains why interpreted serverless functions can have a higher cold-start cost. A JIT compiler works between these two approaches by compiling frequently used code into optimized machine code while the program is running. This can improve performance after warm-up, but it adds runtime, memory, and startup overhead. A CubeSat would generally avoid a JIT because it needs fast, predictable startup and has very limited memory and flash space.

Language	Size (bytes)	Avg. Cold-Start (ms)
Python	1,438	47.59
C	110,727	9.15
Go	2,535,424	14.92